

ACM-XCPC

slmhh

代码模版库



湖北工业大学
HUBEI UNIVERSITY OF TECHNOLOGY

Template For XCPC

slmhh @ HBUT

November 24, 2023

Contents

1	数据结构	3
1.1	单调队列	3
1.2	单调栈	4
1.3	哈希表	4
1.4	ST 表	5
1.5	线段树	6
1.6	树状数组	7
1.7	可持久化线段树 (主席树)	8
1.8	扩展欧几里得算法	10
1.9	欧拉函数	10
1.10	乘法逆元	10
2	字符串	11
2.1	字符串哈希	11
2.2	字典树	11
3	图论	12
3.1	拓扑排序	12
3.2	Dijkstra 算法	12
3.3	Floyd 算法	14

1 数据结构

1.1 单调队列

```
1  #include<iostream>
2  #include<stdio.h>
3  #include<queue>
4  using namespace std;
5
6  const int N = 1e6 + 10;
7  deque<long long int> Q;//储存的是编号
8  long long int a[N];
9  long long int n,k;
10
11 //求最大构建递减的单调队列
12 void cmax(){
13     //创建初始的窗口(到k - 1)
14     for(int i = 0;i < k - 1;i++){
15         //队尾不为空,且队尾元素比a[i]小,则弹出队尾元素
16         while(!Q.empty() && a[i] >= a[Q.back()]) Q.pop_back();
17         Q.push_back(i);
18     }
19
20     for(int i = k - 1;i < n;i++){
21         //队尾不为空,且队尾元素比a[i]小,则弹出队尾元素
22         while(!Q.empty() && a[i] >= a[Q.back()]) Q.pop_back();
23         Q.push_back(i);
24         //窗口滑过了队头元素,则弹出队头元素
25         while(!Q.empty() && Q.front() <= i - k) Q.pop_front();
26         printf("%11d ",a[Q.front()]);
27     }
28     printf("\n");
29 }
30
31 //求最小构建递增的单调队列
32 void cmin(){
33     //创建初始的窗口(到k - 1)
34     for(int i = 0;i < k - 1;i++){
35         while(!Q.empty() && a[i] <= a[Q.back()]) Q.pop_back();
36         Q.push_back(i);
37     }
38
39     for(int i = k - 1;i < n;i++){
40         while(!Q.empty() && a[i] <= a[Q.back()]) Q.pop_back();
41         Q.push_back(i);
42         while(!Q.empty() && Q.front() <= i - k) Q.pop_front();
43         printf("%11d ",a[Q.front()]);
44     }
45     printf("\n");
46 }
47
48 int main(){
49     cin >> n >> k;
```

```
50     for(int i = 0;i < n;i++) cin >> a[i];
51     cmin();
52     Q.clear();
53     cmax();
54     return 0;
55 }
```

1.2 单调栈

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  //P5788 【模板】单调栈
5  typedef long long ll;
6  const int N = 1e6 + 10;
7
8  ll nums[N],ans[N];
9  stack<ll> up;
10
11 int main(){
12     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
13     ll n;
14     cin >> n;
15     for(int i = 0;i < n;i++) cin >> nums[i];
16     for(int i = n - 1;i >= 0;i--){
17         while(!up.empty() && nums[i] >= nums[up.top()]) up.pop();
18         if(up.empty()) ans[i] = 0;
19         else ans[i] = up.top() + 1;
20         up.push(i);
21     }
22     for(int i = 0;i < n;i++) cout << ans[i] << " ";
23     return 0;
24 }
```

1.3 哈希表

```
1  #include<iostream>
2  #include<string.h>
3  using namespace std;
4
5  const int N = 100003;
6
7  int h[N],e[N],ne[N],idx = 0;
8
9  void insert(int x){
10     int k = (x % N + N) % N;
11     e[idx] = x;
12     ne[idx] = h[k];
13     h[k] = idx++;
14 }
15
```

```

16 bool find(int x){
17     int k = (x % N + N) % N;
18     for(int i = h[k]; i != -1; i++){
19         if(e[i] == x)
20             return true;
21     }
22     return false;
23 }
24
25 int main(){
26     int n;
27     scanf("%d",&n);
28     memset(h,-1,sizeof(h));
29     while(n--){
30         char op[2];
31         int x;
32         scanf("%s%d",op,&x);
33         if(op == "1") insert(x);
34         else{
35             if(find(x)) puts("Yes\n");
36             else puts("No\n");
37         }
38     }
39 }

```

1.4 ST 表

ST 表是用于解决可重复贡献问题的数据结构, 如 $\max(x, y)$, $\gcd(a, b)$ 等。

```

1  #include<bits/stdc++.h>
2  #define all(x) (x).begin(),(x).end()
3  using namespace std;
4  typedef long long ll;
5  const int N = 1e5 + 10;
6
7  //P3865 【模板】ST 表
8  //静态区间最大值
9  ll st[N][20],n,m;
10 //opt需要满足幂等律和结合律
11 ll opt(const ll a,const ll b){return max(a,b);}
12
13 ll query(ll l,ll r){
14     ll k = log2(r - l + 1);
15     return opt(st[l][k],st[r - (1 << k) + 1][k]);
16 }
17
18 void build_st(){
19     for(int j = 1;j <= 20;j++){
20         for(int i = 1;i + (1 << j) - 1 <= n;i++){
21             st[i][j] = opt(st[i][j - 1],st[i + (1 << (j - 1))][j - 1]);
22         }
23     }
24
25 int main(){

```

```

25     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
26     cin >> n >> m;
27     for(int i = 1;i <= n;i++){
28         cin >> st[i][0];
29     }
30     build_st();
31     while(m--){
32         ll l,r;
33         cin >> l >> r;
34         cout << query(l,r) << "\n";
35     }
36     return 0;
37 }

```

1.5 线段树

线段树可以在 $O(\log N)$ 的时间复杂度内实现单点修改、区间修改、区间查询（区间求和，求区间最大值，求区间最小值）等操作。

```

1  #include <iostream>
2  typedef long long LL,ll;
3  LL n, a[100005]; //原数组
4  LL d[270000]; //线段树
5  LL b[270000]; //懒惰标记
6
7
8  //P3372 【模板】线段树 1
9  void build(LL l, LL r, LL p) { // l:区间左端点 r:区间右端点 p:节点标号
10     if (l == r) { //如果区间长为1
11         d[p] = a[l]; //将节点赋值 a下标是l!!!!
12         return;
13     }
14     LL m = l + ((r - l) >> 1); //取区间中点
15     build(l, m, p << 1), build(m + 1, r, (p << 1) | 1); //分别建立左右子树 第一个是字母
16     d[p] = d[p << 1] + d[(p << 1) | 1]; //求和
17 }
18
19 void update(LL l, LL r, LL c, LL s, LL t, LL p) { // s:要修改区间的左端点 t:要修改区间的
20     //左端点 c:加上(更新)的值
21     if (l <= s && t <= r) {
22         d[p] += (t - s + 1) * c; // 如果区间被包含了，直接得出答案
23         b[p] += c; //懒惰标记
24         return;
25     }
26     LL m = s + ((t - s) >> 1); //取区间中点
27     if (b[p]){ //如果有懒惰标记
28         d[p << 1] += b[p] * (m - s + 1); //左结点的区间长度乘c
29         d[(p << 1) | 1] += b[p] * (t - m); //右结点的区间长度乘c
30         b[p << 1] += b[p]; //左结点添加懒惰标记
31         b[(p << 1) | 1] += b[p]; //右结点添加懒惰标记
32     }
33     b[p] = 0; //懒惰标记归零

```

```

33     if (l <= m)
34         update(l, r, c, s, m, p << 1); //本行和下面的一行用来更新p*2和p*2+1的节点
35     if (r > m)
36         update(l, r, c, m + 1, t, (p << 1) | 1);
37     d[p] = d[p << 1] + d[(p << 1) | 1]; //更新该节点区间和
38 }
39
40 LL getsum(LL l, LL r, LL s, LL t, LL p) {
41     if (l <= s && t <= r) return d[p];
42     LL m = s + ((t - s) >> 1);
43     if (b[p]){
44         d[p << 1] += b[p] * (m - s + 1); //同update
45         d[(p << 1) | 1] += b[p] * (t - m);
46         b[p << 1] += b[p];
47         b[(p << 1) | 1] += b[p];
48     }
49     b[p] = 0;
50     LL sum = 0;
51     if (l <= m)
52         sum = getsum(l, r, s, m, p << 1); // 本行和下面的一行用来更新p*2和p*2+1的答案
53     if (r > m)
54         sum += getsum(l, r, m + 1, t, (p << 1) | 1);
55     return sum;
56 }
57
58 int main() {
59     std::ios::sync_with_stdio(0);
60     LL q, op, i2, i3, i4;
61     std::cin >> n >> q;
62     for (LL i = 1; i <= n; i++) std::cin >> a[i];
63     build(1, n, 1); //从1开始建立线段树
64     while (q--) {
65         std::cin >> op >> i2 >> i3;
66         if (op == 2)
67             std::cout << getsum(i2, i3, 1, n, 1) << std::endl; // 直接调用操作函数
68         else
69             std::cin >> i4, update(i2, i3, i4, 1, n, 1);
70     }
71     return 0;
72 }

```

1.6 树状数组

树状数组是一种支持单点修改和区间查询的，代码量小的数据结构。普通树状数组维护的信息及运算要满足结合律且可差分，如加法（和）、乘法（积）、异或等。

```

1 //区间加区间和
2 int t1[MAXN], t2[MAXN], n;
3
4 int lowbit(int x) { return x & (-x); }
5
6 void add(int k, int v) {
7     int v1 = k * v;

```

```

8   while (k <= n) {
9       t1[k] += v, t2[k] += v1;
10      // 注意不能写成 t2[k] += k * v, 因为 k 的值已经不是原数组的下标了
11      k += lowbit(k);
12  }
13  }
14
15  int getsum(int *t, int k) {
16      int ret = 0;
17      while (k) {
18          ret += t[k];
19          k -= lowbit(k);
20      }
21      return ret;
22  }
23
24  void add1(int l, int r, int v) {
25      add(l, v), add(r + 1, -v); // 将区间加差分分为两个前缀加
26  }
27
28  long long getsum1(int l, int r) {
29      return (r + 1ll) * getsum(t1, r) - 1ll * l * getsum(t1, l - 1) -
30             (getsum(t2, r) - getsum(t2, l - 1));
31  }

```

1.7 可持久化线段树 (主席树)

给定 n 个整数构成的序列 a , 将对于指定的闭区间 $[l, r]$ 查询其区间内的第 k 小值。

```

1  #include <algorithm>
2  #include <cstdio>
3  #include <cstring>
4  using namespace std;
5  const int maxn = 1e5; // 数据范围
6  int tot, n, m;
7  int sum[(maxn << 5) + 10], rt[maxn + 10], ls[(maxn << 5) + 10],
8      rs[(maxn << 5) + 10];
9  int a[maxn + 10], ind[maxn + 10], len;
10
11  int getid(const int &val) { // 离散化
12      return lower_bound(ind + 1, ind + len + 1, val) - ind;
13  }
14
15  int build(int l, int r) { // 建树
16      int root = ++tot;
17      if (l == r) return root;
18      int mid = l + r >> 1;
19      ls[root] = build(l, mid);
20      rs[root] = build(mid + 1, r);
21      return root; // 返回该子树的根节点
22  }
23
24  int update(int k, int l, int r, int root) { // 插入操作

```

```

25     int dir = ++tot;
26     ls[dir] = ls[root], rs[dir] = rs[root], sum[dir] = sum[root] + 1;
27     if (l == r) return dir;
28     int mid = l + r >> 1;
29     if (k <= mid)
30         ls[dir] = update(k, l, mid, ls[dir]);
31     else
32         rs[dir] = update(k, mid + 1, r, rs[dir]);
33     return dir;
34 }
35
36 int query(int u, int v, int l, int r, int k) { // 查询操作
37     int mid = l + r >> 1,
38         x = sum[ls[v]] - sum[ls[u]]; // 通过区间减法得到左儿子中所存储的数值个数
39     if (l == r) return l;
40     if (k <= x) // 若 k 小于等于 x, 则说明第 k 小的数字存储在左儿子中
41         return query(ls[u], ls[v], l, mid, k);
42     else // 否则说明在右儿子中
43         return query(rs[u], rs[v], mid + 1, r, k - x);
44 }
45
46 void init() {
47     scanf("%d%d", &n, &m);
48     for (int i = 1; i <= n; ++i) scanf("%d", a + i);
49     memcpy(ind, a, sizeof ind);
50     sort(ind + 1, ind + n + 1);
51     len = unique(ind + 1, ind + n + 1) - ind - 1;
52     rt[0] = build(1, len);
53     for (int i = 1; i <= n; ++i) rt[i] = update(getid(a[i]), 1, len, rt[i - 1]);
54 }
55
56 int l, r, k;
57
58 void work() {
59     while (m--) {
60         scanf("%d%d%d", &l, &r, &k);
61         printf("%d\n", ind[query(rt[l - 1], rt[r], 1, len, k)]); // 回答询问
62     }
63 }
64
65 int main() {
66     init();
67     work();
68     return 0;
69 }
70 \end{lstlisting}
71
72 \section{数学}
73 \subsection{欧拉筛}
74 \begin{lstlisting}[caption={}]
75 bool isprime[N + 10];
76 int prime[N + 10], cnt = 0, temp;
77

```

```

78 void eular() {
79     memset(isprime, true, sizeof(isprime));
80     isprime[1] = false;
81     for (int i = 2; i <= N; i++) {
82         if (isprime[i]) prime[++cnt] = i;
83         for (int j = 1; j <= cnt && i * prime[j] <= N; j++) {
84             isprime[i * prime[j]] = false;
85             if (i % prime[j] == 0) break;
86         }
87     }
88 }

```

1.8 扩展欧几里得算法

```

1 int exgcd(int a,int b,int &x,int &y){
2     if(b == 0){
3         x = 1;
4         y = 0;
5         return a;
6     }
7     int d = exgcd(b,a % b,y,x);
8     y -= (a / b) * x;
9     return d;
10 }

```

1.9 欧拉函数

$$\varphi(n) = n \times \prod_{i=1}^s \frac{p_i - 1}{p_i}.$$

```

1 #include <cmath>
2
3 int euler_phi(int n) {
4     int ans = n;
5     for (int i = 2; i * i <= n; i++)
6         if (n % i == 0) {
7             ans = ans / i * (i - 1);
8             while (n % i == 0) n /= i;
9         }
10    if (n > 1) ans = ans / n * (n - 1);
11    return ans;
12 }

```

1.10 乘法逆元

如果一个线性同余方程 $ax \equiv 1 \pmod{b}$, 则 x 称为 $a \bmod b$ 的逆元, 记作 a^{-1} 。

```

1 inv[1] = 1;
2 printf("%d\n", inv[1]);
3 for(int i = 2; i <= n; i++){
4     inv[i] = (long long)(p - p / i) * inv[p % i] % p;
5     printf("%d\n", inv[i]);
6 }

```

2 字符串

2.1 字符串哈希

```
1 int count_unique_substrings(string const& s) {
2     int n = s.size();
3
4     const int b = 31;
5     const int m = 1e9 + 9;
6     vector<long long> b_pow(n);
7     b_pow[0] = 1;
8     for (int i = 1; i < n; i++) b_pow[i] = (b_pow[i - 1] * b) % m;
9
10    vector<long long> h(n + 1, 0);
11    for (int i = 0; i < n; i++)
12        h[i + 1] = (h[i] + (s[i] - 'a' + 1) * b_pow[i]) % m;
13
14    int cnt = 0;
15    for (int l = 1; l <= n; l++) {
16        set<long long> hs;
17        for (int i = 0; i <= n - l; i++) {
18            long long cur_h = (h[i + l] + m - h[i]) % m;
19            cur_h = (cur_h * b_pow[n - i - 1]) % m;
20            hs.insert(cur_h);
21        }
22        cnt += hs.size();
23    }
24    return cnt;
25 }
```

2.2 字典树

```
1 struct trie {
2     int nex[100000][26], cnt;
3     bool exist[100000]; // 该结点结尾的字符串是否存在
4
5     void insert(char *s, int l) { // 插入字符串
6         int p = 0;
7         for (int i = 0; i < l; i++) {
8             int c = s[i] - 'a';
9             if (!nex[p][c]) nex[p][c] = ++cnt; // 如果没有，就添加结点
10            p = nex[p][c];
11        }
12        exist[p] = 1;
13    }
14
15    bool find(char *s, int l) { // 查找字符串
16        int p = 0;
17        for (int i = 0; i < l; i++) {
18            int c = s[i] - 'a';
19            if (!nex[p][c]) return 0;
20            p = nex[p][c];
21        }
22        return exist[p];
23    }
24 }
```

```
21     }
22     return exist[p];
23 }
24 };
```

3 图论

3.1 拓扑排序

```
1  int n, m;
2  vector<int> G[MAXN];
3  int in[MAXN]; // 存储每个结点的入度
4
5  bool toposort() {
6      vector<int> L;
7      queue<int> S;
8      for (int i = 1; i <= n; i++)
9          if (in[i] == 0) S.push(i);
10     while (!S.empty()) {
11         int u = S.front();
12         S.pop();
13         L.push_back(u);
14         for (auto v : G[u]) {
15             if (--in[v] == 0) {
16                 S.push(v);
17             }
18         }
19     }
20     if (L.size() == n) {
21         for (auto i : L) cout << i << ' ';
22         return true;
23     } else {
24         return false;
25     }
26 }
```

3.2 Dijkstra 算法

```
1  #include<iostream>
2  #include<stdio.h>
3  #include<string.h>
4  #include<string>
5  #include<algorithm>
6  #include<queue>
7  using namespace std;
8
9  //无负权边
10 typedef pair<int,int> PII;
11 const int N = 1e6 + 10;
12
13 struct edge{
```

```
14     int weight; //权重
15     int next; //邻接表的下一项
16     int to; //指向的边
17 }e[N];
18
19 int head[N]; //头节点为位置
20 int dist[N]; //节点1到起点的距离
21 bool st[N]; //st[i]表示该节点是否确定了最小距离, 1为true, 0为false
22 int n,m; //n个点, m条边
23 int idx = 0;
24 int start; //起点
25
26 //边:a->b,权重为c
27 void add(int a,int b,int c){
28     e[idx].to = b; //指向的边
29     e[idx].next = head[a];
30     e[idx].weight = c; //权重
31     head[a] = idx; //插入
32     idx++;
33 }
34
35
36 void dijkstra(){
37     priority_queue <PII,vector<PII>,greater<PII> > heap;
38     heap.push({0,start}); //first为距离, second为节点
39
40     memset(dist,0x3f,sizeof(dist)); //把距离初始化为正无穷
41     dist[start] = 0; //起点为1
42
43     while(heap.size()){
44         PII t = heap.top();
45         heap.pop();
46
47         int ver = t.second,d = t.first;
48         if(st[ver] == 1) continue;
49         st[ver] = 1;
50
51         //a -> b,已知a的距离和边权,去更新b的距离
52         for(int i = head[ver];i != -1;i = e[i].next){
53             int j = e[i].to; //指向的边
54             if(dist[j] > d + e[i].weight){
55                 dist[j] = d + e[i].weight;
56                 heap.push({dist[j],j});
57             }
58         }
59     }
60 }
61
62 int main(){
63
64     cin >> n >> m >> start;
65     memset(head,-1,sizeof(head));
66     while(m--){
```

```
67     int x,y,z;
68     cin >> x >> y >> z;
69     add(x,y,z);
70 }
71 dijkstra();
72 for(int i = 1;i <= n;i++){
73     cout << dist[i] << " ";
74 }
75 /*if(dist[n] == 0x3f3f3f3f)
76     cout << "-1\n";
77 else
78     cout << dist[n] << endl;
79 return 0;*/
80 }
```

3.3 Floyd 算法

```
1  int n,m,k,x,y,z;
2  int dist[N][M];
3
4  //d[i][j] 表示点i到点j的最短距离 ,k为中间点
5  void floyd(){
6      for(k = 1;k <= n;k++){ //枚举中间点
7          for(int i = 1;i <= n;i++){ //起始点
8              for(int j = 1;j <= n;j++){ //目的地
9                  dist[i][j] = min(dist[i][j] , dist[i][k] + dist[k][j]);
10             }
11         }
12     }
13 }
```